SOFTWARE REQUIREMENTS SPECIFICATION

Autonomous Video Ad-Placement & Contextual Safety Analyzer

Submitted in partial fulfillment of the requirements for the course

Specialization Project

Submitted By

Andrew Shelton.S	2441609
Akash Ganesh	2441604
Andrew Judah	2441608
Ramona Elisha Mary	2441646

Under the Guidance of

Dr.Chanti

Assistant Professor

Department of Computer Science

School of Sciences

CHRIST (Deemed to be University)

Bangalore

Academic Year 2026–2027

Contents

1 Introduction **4**

1.1 Purpose 4

1.2 Scope 4

1.3 Intended Audience 4

1.4 Definitions and Acronyms 4

2 Overall Description **5**

2.1 Product Perspective 5

2.2 Product Functions 5

2.3 User Characteristics 6

2.4 Assumptions and Dependencies 6

3 Functional Requirements **6**

3.1 FR1: Video Ingestion & Controller Pipeline (Module 3) 6

3.2 FR2: Audio & NLP Auditing Engine (Module 1) 6

3.3 FR3: Vision & Ad-Spot Detection Engine (Module 2) 7

3.4 FR4: Interactive Dashboard & Timeline UI (Module 4) 7

4 Non-Functional Requirements **7**

5 Use Case Description **8**

5.1 Use Case 1: Automated Video Auditing & Ad-Spot Analysis 8

5.2 Use Case 2: Compliance Report Export 9

6 System Architecture **9**

7 Database Design **11**

7.1 Video Metadata Table 11

7.2 Audio Flags Table 11

7.3 Visual Flags Table 12

7.4 Ad Spots Table 12

8 External Interface Requirements **12**

8.1 User Interface 12

8.2 Hardware Interface 13

9 Software and Hardware Requirements **13**

9.1 Software Requirements 13

9.2 Hardware Requirements 13

10 References **14**

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly define the functional and non-functional requirements for the **Autonomous Video Ad-Placement & Contextual Safety Analyzer**. It serves as a blueprint for design, development, testing, and deployment.

1.2 Scope

The proposed software is a localized, offline desktop application designed to assist digital marketing agencies and content creators in auditing long-form videos. Utilizing localized multimodal AI (Computer Vision and Natural Language Processing), the system scans videos to flag compliance and brand safety risks (such as profanity, sensitive discussions, weapons, or smoking) to prevent demonetization. Simultaneously, it calculates spatial geometry to identify premium non-intrusive ad placement spots (e.g., blank walls, laptop screens, empty desks) suitable for digital sponsor overlays.

1.3 Intended Audience

This document is intended for project advisors, developers, quality assurance testers, software architects, and evaluators.

1.4 Definitions and Acronyms

- **AI** – Artificial Intelligence
- **CV** – Computer Vision
- **NLP** – Natural Language Processing
- **STT** – Speech-to-Text
- **YOLO** – You Only Look Once (Object Detection Model)
- **FPS** – Frames Per Second

- **CRUD** – Create, Read, Update, Delete
- **UI** – User Interface
- **SRS** – Software Requirements Specification

2. Overall Description

2.1 Product Perspective

The system is an independent, offline desktop application constructed using Python 3.10+, SQLite3 for relational data storage, and Streamlit for an interactive user dashboard. It integrates local AI engines (OpenAI Whisper and Ultralytics YOLOv8) without relying on external cloud APIs, ensuring data privacy and zero ongoing API costs.

2.2 Product Functions

The core capabilities of the application include:

1. **Video Ingestion & Sampling:** Uploading MP4/AVI video files and extracting frames at 1 FPS alongside isolated audio tracks.
2. **Audio Compliance Auditing:** Local transcription via Whisper and keyword/regex pattern matching to flag restricted words and timestamps.
3. **Visual Risk Detection:** Object detection using YOLOv8 to flag visual violations (e.g., weapons, smoking, illicit substances).
4. **Ad-Spot Spatial Calculation:** Bounding box geometry analysis to detect continuous empty surfaces suitable for sponsor placement overlays.
5. **Central Database Orchestration:** Recording video metadata, timestamped flags, and ad coordinates in an SQLite database.
6. **Interactive UI Dashboard:** Timeline visualization of flagged events, synchronized video playback, data filtering, and compliance report export (PDF/CSV).

2.3 User Characteristics

- **Digital Marketing Agencies:** Require rapid auditing of creator content before placing sponsor brands.
- **Video Creators / Editors:** Need to verify brand safety and monetization eligibility before platform upload.
- **Compliance Officers:** Ensure regulatory adherence for digital media broadcasts.

2.4 Assumptions and Dependencies

- The host machine possesses sufficient multi-core CPU or hardware acceleration (CUDA/GPU) to run local YOLOv8 and Whisper inference.
- Input videos are in standard digital formats (MP4, MOV, AVI) with clear audio tracks.
- FFmpeg is installed and accessible via system PATH for MoviePy audio extraction.

3. Functional Requirements

3.1 FR1: Video Ingestion & Controller Pipeline (Module 3)

- The system shall allow users to upload video files via the Streamlit dashboard.
- The pipeline orchestrator shall generate unique video records in SQLite containing filepath, duration, resolution, and processing status.

3.2 FR2: Audio & NLP Auditing Engine (Module 1)

- The system shall extract audio streams from uploaded videos using MoviePy.
- The system shall run local Speech-to-Text transcription using OpenAI Whisper.
- The system shall match transcription segments against a customizable dictionary of restricted keywords and profanity, logging exact start/end timestamps and severity levels into the database.

3.3 FR3: Vision & Ad-Spot Detection Engine (Module 2)

- The system shall sample video frames at a rate of 1 frame per second (1 FPS) using OpenCV.
- The system shall execute YOLOv8 inference to detect predefined visual risks (e.g., weapons, alcohol, smoking) and store confidence scores and bounding boxes.
- The system shall compute continuous planar area dimensions to identify prime ad placement zones (blank walls, screens) exceeding threshold pixel dimensions.

3.4 FR4: Interactive Dashboard & Timeline UI (Module 4)

- The system shall present an interactive dashboard featuring a video player integrated with clickable timestamp flags.
- The UI shall display tabular summaries of audio flags, visual risks, and recommended ad spots fetched dynamically from SQLite.
- The user shall be able to export full compliance audit reports in CSV and PDF formats.

4. Non-Functional Requirements

1. **Offline Privacy:** All AI processing must occur locally without transmitting video or audio data to external third-party servers.
2. **Performance:** Frame sampling at 1 FPS and audio transcription should complete within a reasonable timeframe (e.g., under 0.5x realtime on GPU hardware).
3. **Accuracy:** Visual risk detection and keyword flagging shall maintain a precision rate greater than 85% on benchmark testing sets.
4. **Reliability:** The database orchestrator shall ensure ACID compliance during concurrent write operations from audio and vision threads.
5. **Usability:** The UI dashboard shall be intuitive, requiring zero technical knowledge of AI models to interpret audit results.

5. Use Case Description

5.1 Use Case 1: Automated Video Auditing & Ad-Spot Analysis

Use Case Name	Automated Video Auditing & Ad-Spot Analysis
Actor	Digital Marketing Auditor / Video Creator
Precondition	Application launched locally via Streamlit
Main Flow	1. User uploads an MP4 video file. 2. Controller initializes DB record and triggers parallel engines. 3. Audio engine transcribes and flags restricted speech. 4. Vision engine scans frames for visual hazards and ad spots. 5. UI updates dashboard with interactive timeline and tables.
Post Condition	Audit data persisted in SQLite; timeline ready for review

5.2 Use Case 2: Compliance Report Export

Use Case Name	Compliance Report Export
Actor	Digital Marketing Auditor
Precondition	Video processing completed successfully
Main Flow	1. User filters flagged events on the dashboard. 2. User clicks the "Export Report (PDF/CSV)" button. 3. System compiles DB records into a structured summary report.
Post Condition	PDF or CSV file downloaded to user's local machine

6. System Architecture

The software architecture follows a modular, layer-based design isolating frontend presentation, pipeline control, AI inference, and storage.

1. **Presentation Layer:** Streamlit UI handling video uploads, interactive timelines, and visual charts.

2. **Controller Layer:** Python pipeline orchestrator managing background task dispatching and state coordination.

3. **AI Engine Layer:** Multimodal processing engines comprising OpenCV/YOLOv8 (Vision) and MoviePy/Whisper (Audio).

4. **Data Access Layer:** SQLite3 relational database maintaining structured audit records.

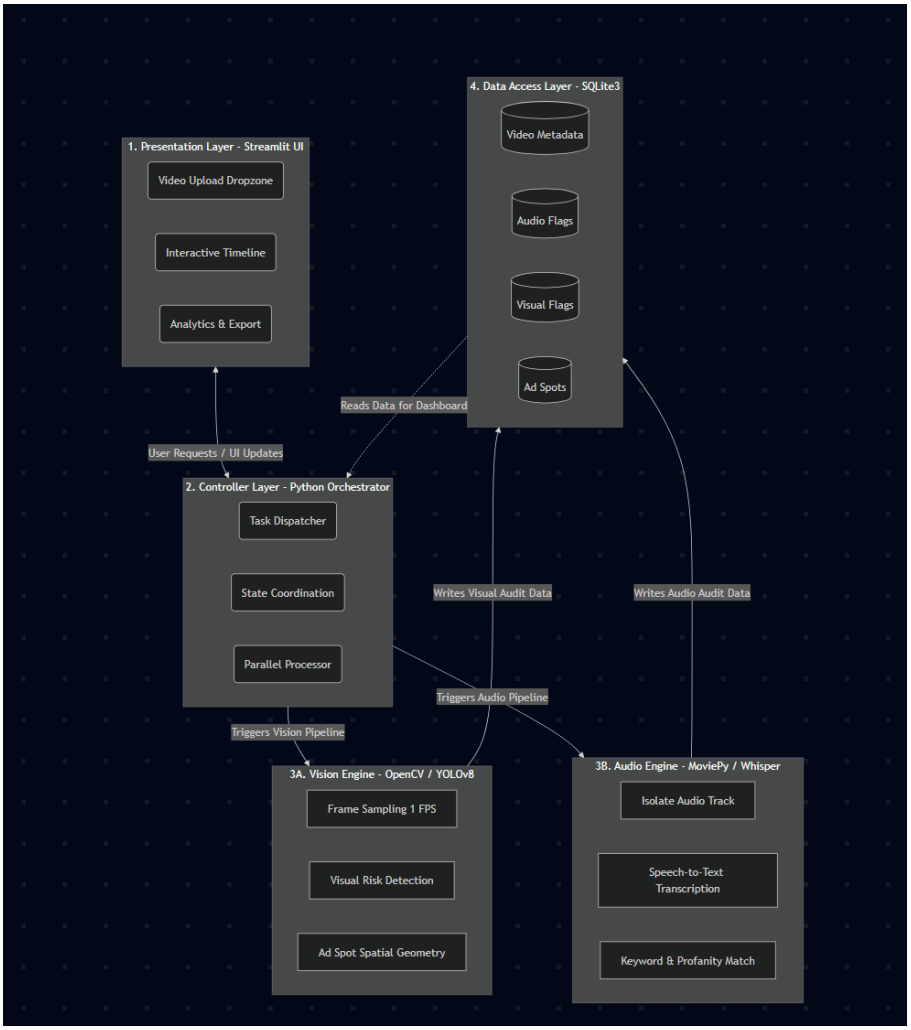


Figure 1: System Architecture

7. Database Design

7.1 Video Metadata Table

Field Name	Data Type	Description
video_id	INTEGER	Primary Key (Auto-increment)
filename	TEXT	Original name of uploaded video
filepath	TEXT	Local storage path
duration_sec	REAL	Total length in seconds
fps	REAL	Frames per second
processed_status	TEXT	Processing status (pending, completed, failed)

7.2 Audio Flags Table

Field Name	Data Type	Description
flag_id	INTEGER	Primary Key (Auto-increment)
video_id	INTEGER	Foreign Key referencing video_metadata
timestamp_start	REAL	Start time of flagged audio (seconds)
timestamp_end	REAL	End time of flagged audio (seconds)
word_flagged	TEXT	Specific restricted word or phrase detected
severity	TEXT	Risk rating (Low, Medium, High)

7.3 Visual Flags Table

Field Name	Data Type	Description
flag_id	INTEGER	Primary Key (Auto-increment)
video_id	INTEGER	Foreign Key referencing video_metadata
timestamp	REAL	Frame timestamp in seconds
object_detected	TEXT	Risk label detected (e.g., weapon, smoking)
bounding_box	TEXT	Coordinates [x1, y1, x2, y2]
confidence	REAL	YOLO detection confidence score

7.4 Ad Spots Table

Field Name	Data Type	Description
spot_id	INTEGER	Primary Key (Auto-increment)
video_id	INTEGER	Foreign Key referencing video_metadata
timestamp_start	REAL	Start time of ad placement opportunity
timestamp_end	REAL	End time of ad placement opportunity
spot_type	TEXT	Surface type (e.g., blank_wall, screen)
bounding_box	TEXT	Coordinates [x1, y1, x2, y2]
area_pixels	REAL	Continuous planar area in pixels

8. External Interface Requirements

8.1 User Interface

The application interface shall be clean, modern, and accessible via any standard web browser running against the local Streamlit port (localhost:8501). Key interfaces include:

- **Upload View:** Drag-and-drop file dropzone with processing progress bars.
- **Audit Dashboard:** Embedded video player synced with color-coded risk timeline bars.
- **Analytics View:** Tabular breakdowns allowing quick jumping to flagged timestamps.

8.2 Hardware Interface

- Local File System interface for reading video streams and storing database artifacts.
- Optional NVIDIA CUDA GPU interface for accelerated PyTorch tensor operations during YOLO and Whisper inference.

9. Software and Hardware Requirements

9.1 Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Programming Language:** Python 3.10+
- **AI/CV Libraries:** PyTorch, Ultralytics (YOLOv8), OpenAI Whisper, OpenCV, MoviePy
- **Frontend & Data:** Streamlit, Pandas, SQLite3
- **Version Control:** Git & GitHub

9.2 Hardware Requirements

- **Processor:** Multi-core CPU (Intel i5/Ryzen 5 or higher)
- **Memory:** Minimum 16 GB RAM recommended for loading multimodal weights
- **Storage:** 10 GB available disk space for model weights and video caching
- **GPU (Optional):** NVIDIA GPU with 6GB+ VRAM for accelerated inference

10. References

1. IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).

2. Ultralytics YOLOv8 Official Documentation. `https://docs.ultralytics.com/`

3. OpenAI Whisper Robust Speech Recognition via Large-Scale Weak Supervision. `https://github.com/openai/whisper`

4. Streamlit Documentation for Web App Development. `https://docs.streamlit.io/`

5. OpenCV (Open Source Computer Vision Library). `https://opencv.org/`